微服务之服务拆分

一、 前言

领域驱动设计（DDD）的普及和应用让微服务拆分和落地有了理论的指导，有章可循，有法可依。特别是在一个产品或者项目开发的初期，可以很自然的通过 DDD 的知识帮助进行微服务的划分，指导架构设计。而在项目迭代开发过程中，架构已经稳定，业务需求源源不断，将已有服务进行拆分的场景并不多见，思考其原因可能有以下两个方面：

1、已有架构可以满足当前的业务需求，服务拆分会产生很多工作量，拆分能带来哪些收益，会解决哪些问题，需要经过慎重的思考才能做出决策

2、业务优先级通常更高，拆分时机很难选择，产品在持续演进，迭代开发在持续的进行，服务拆分工作繁杂，很难在一个跌代里全部完成，一方面有限的资源要优先保证业务需求正常上线，另一方面拆分工作和业务开发同时进行，既要保证业务的稳定也要顺利推进拆分工作，拆分时机并不太好把握

此次分享，根据之前项目中遇到的服务拆分问题和实践，简单浅析一下微服务拆分必要性、拆分规范和拆分时机等问题

二、 微服务拆分必要性（不要为了拆分而拆分）

服务拆分出发点一定是为了解决项目开发过程中的某些痛点和问题，而不是仅仅从服务划分的合理性上来做出拆分决定（特别是在产品已经相对稳定，新的需求不断开发过程中）下面，我们通过以下两点来思考拆分服务的必要性：

1、解耦业务，降低上下文理解难度，追求更合理的产品设计

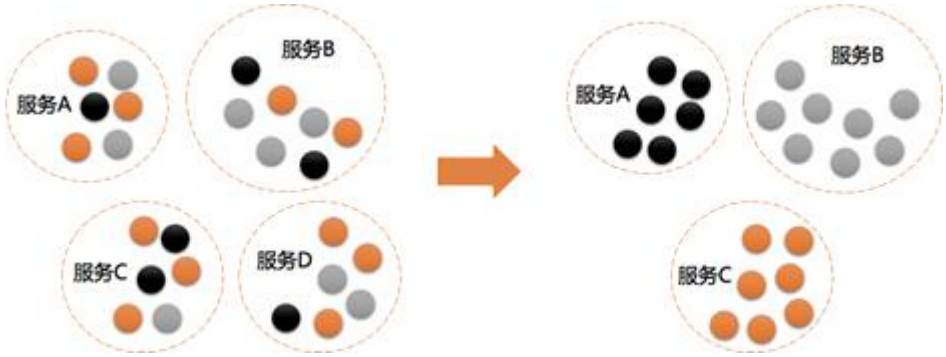
2、降低系统复杂度，提升开发效率，提升项目整体质量。

三、 微服务拆分规范

微服务拆分之后，工程项目会比较的多，如果没有一定的规范约束，将会非常混乱，难以维护，反而增加了项目的复杂度。

1、高内聚、低耦合

紧密关联的事物应该放在一起，每个服务是针对一个单一职责的业务能力的封装，专注做好一件事情。如下图：有四个服务 A、B、C、D，但是每个服务职责不单一，A 可能在做 B 的事情，B 又在做 C 的事情，C 又同时在做 A 的事情，通过重新调整，将相关的事物放在一起后，可以减少不必要的服务。



通用拆分方式：

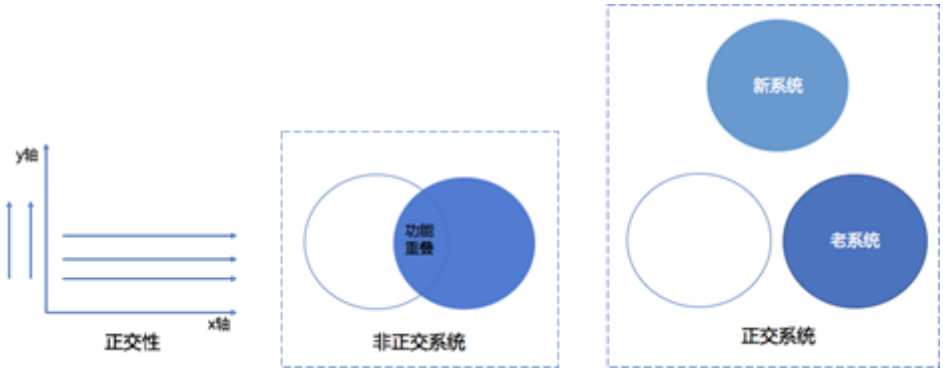
先按业务领域拆（垂直），如社区、用户、商城、慢病、工单，如果有相同功能需要聚合，则进行下沉。

再按功能定位拆（水平），如商城业务复杂度提高之后可进一步拆分为商品、订单、物流、支付。

按重要程度拆，区分核心与非核心，如订单核心，订单非核心。

2、服务拆分正交性原则

两条直线相交成直角，就是正交的。正交也就是两条直线互不依赖。如果一个系统的变化不影响另一个系统这些系统就是正交的。



3、服务拆分层级最多三层，两次调用

服务拆分是为了横向扩展，因而应该横向拆分，而非纵向拆成一串的。也即应该将商品和订单拆分，而非下单的十个步骤拆分，然后一个调用一个。这里纵向的拆分最多三层，大部分情况下只有两次调用

1、基础服务层：用于屏蔽数据库，缓存层，提供原子的对象查询接口，有这一层，为了数据层做一定改变的时候，例如分库分表，数据库扩容，缓存替换等，对于上层透明，上层仅仅调用这一层的接口，不直接访问数据库和缓存。

2、组合服务层：这一层调用基础服务层，完成较为复杂的业务逻辑，实现分布式事务也多在这一层

3、Controller 层：接口层，调用组合服务层对外

4、服务粒度适中、演进式拆分

微服务拆分并不是一步到位的，应当根据实际情况逐步展开。如果一开始不知道应该划分多细，完全可以先粗粒度划分，然后随着需要，初步拆分。比如一个电商一开始索性可以拆分为商品服务和交易服务，一个负责展示商品，一个负责购买支付。随后随着交易服务

越来越复杂，就可以逐步的拆分成订单服务和支付服务。

此外，一个微服务需要足够简单，站在微服务角度而言往往只需要 2 人左右可方便快速维护。如果维护的人员过多，要么这个服务过于复杂成为了单体应用；要么是服务边界划分得不够明确；要么是人员组织架构的职责不清。

5、避免环形依赖、双向依赖

服务之间的环形/双向依赖会使得服务间耦合加重，在服务升级的的时候会比较头疼，不知道应该先升级哪个，后升级哪个，难以维护。因此我们需要极力避免服务间的这种复杂依赖关系。

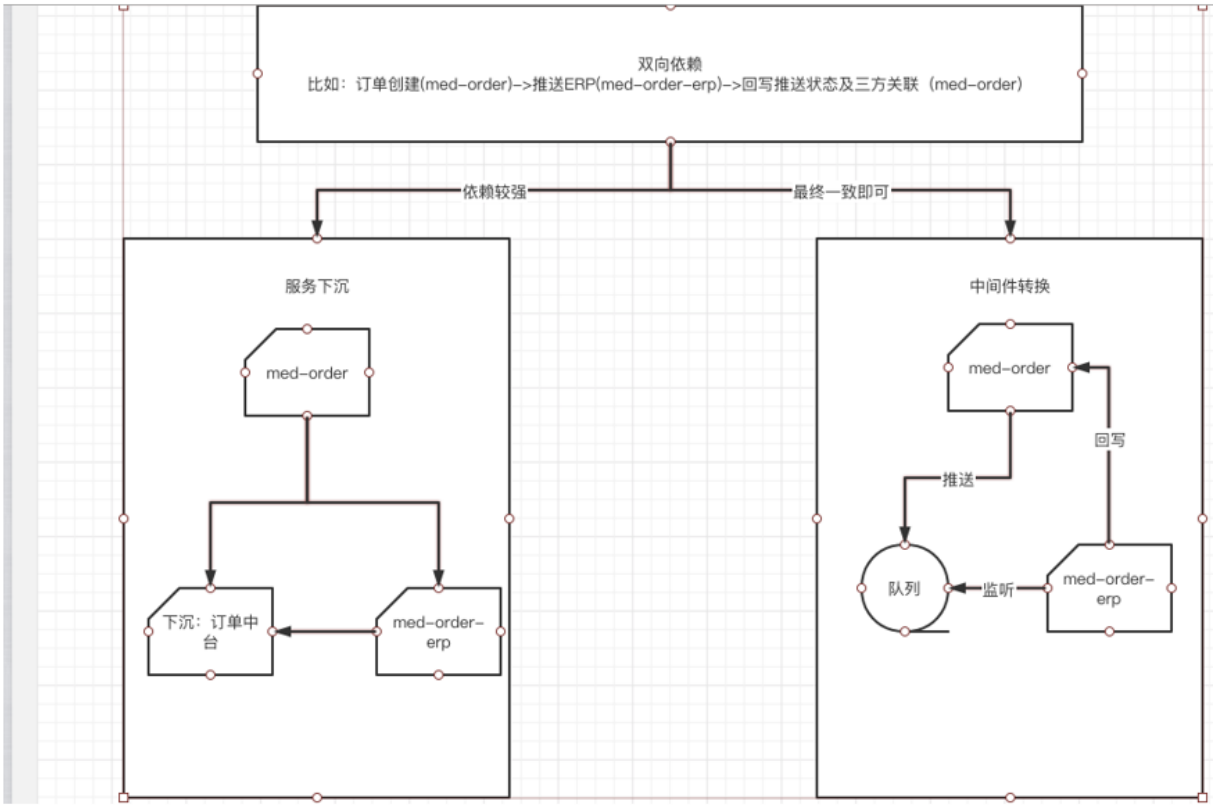
循环依赖

双向依赖

解决这种复杂依赖关系，我们有两种服务拆分避免方式，根据业务场景合理选择：

1、将共同依赖的服务功能进行下沉，拆分为第三方服务。还要注意下沉的服务维护方需要多方协商确定维护者。

2、如果在业务流程上允许异步解耦的，可以考虑引入消息中间件来处理，消息中间件也是微服务治理中使用较多的核心组件。



6、通用化接口设计，减少定制化设计

提微服务提供的服务一定是尽可能通用的，面向功能来开发的，而不是面向调用方来开发的。比如某个调用方提出了一个需求：调用方 B 希望 A 服务提供一个获取订单列表的接口，那么 A 服务设计的接口就应该是 GetOrderList()，而不是 GetOrderListForA()。

7、将串行调用改为并行调用，或者做异步化

如果有的组合服务处理流程的确很长，需要调用多个外部服务，应该考虑如何通过消息队列，实现异步化和解耦。

例如下单之后，要刷新缓存，要通知仓库等，这些都不需要再下单成功的时候就要做完，而是可以发一个消息给消息队列，异步通知其他服务。

而且使用消息队列的好处是，你只要发送一个消息，无论下游依赖方有一个，还是有十个，都是一条消息搞定，只不过多几个下游监听消息即可。

对于下单必须同时做完的，例如扣减库存和优惠券等，可以进行并行调用，这样处理时间会大大缩短，不是多次调用的时间之和，而是最长的那个系统调用时间。

8、接口应该实现幂等性

微服务拆分之后，服务之间的调用当出现错误的时候，往往都会重试，但是为了不要下两次单，支付两次，微服务接口应当实现幂等性。

幂等操作使用状态机，当一个调用到来的时候，往往触发一个状态的变化，当下次调用到来的时候，发现已经不是这个状态，就说明上次已经调用过了。状态的变化需要是一个原子操作，也即并发调用的时候，只有一次可以执行。幂等操作状态机统一使用分布式锁中间件来实现。

9、接口数据定义严禁内嵌，透传

微服务接口之间传递数据，往往通过数据结构，如果数据结构透传，从底层一直到上层使用同一个数据结构，或者上层的数据结构内嵌底层的数据结构，当数据结构中添加或者删除一个字段的时候，波及的面会非常大。

因而接口数据定义，在每两个接口之间约定，严禁内嵌和透传，即便差不多，也应该重新定义，这样接口数据定义的改变，影响面仅仅在调用方和被调用方，当接口需要更新的时候，比较可控，也容易升级。

10、避免服务间共享数据库

数据库包括任意的数据存储服务，例如：MySQL、Redis、MongoDB 等。如果服务间共享数据库，会造成：

1、强耦合：为多个服务提供单个数据库会造成服务间紧密耦合，也会造成服务独立部署困难。

2、扩展性差：使用这种设计很难扩展单个服务，因为这样只能选择扩展整个单个数据库。

3、性能问题：使用一个共享数据库，在一段时间内，你可能最终会得到一个巨大的表。

因此，对于现有共享庞大数据库的微服务，建议是按照业务维度拆分成多个小的数据库，分开独立维护。禁止跨库联表查询。

11、同时应当考虑团队结构

前面讲的都是技术因素的划分原则，其实微服务拆分时也应当考虑团队组织结构。

1、团队足够轻量级，2 pizza 原则，保证团队内部能够高效沟通。

2、团队的职责足够明确，保证能够独立维护，减少团队间工作耦合度，降低跨团队协作成本。

四、微服务拆分时机

微服务拆分绝非一个大跃进运动，由高层发起，把一个应用拆分的七零八落的，最终大大增加运维成本，但是并不会带来收益。微服务拆分的过程，应该是一个由痛点驱动的，是业务真正遇到了快速迭代和高并发的问题，如果不拆分，将对于业务的发展带来影响，只有这个时候，微服务的拆分是有确定收益的，增加的运维成本才是值得的。

产品在不断的演进，开发工作也是每个迭代在持续的进行，必须保证拆分工作不影响日常的业务功能开发；拆分服务本身工作量很大，每个服务都有自己的特殊性，没有一套通用的方案可以直接实施，需要在一步一步的拆分中仔细分析，找到下一步的关键问题并针对解决，这不是一个短期的工作，准确说没有办法在一个迭代周期内完成，所以必须保证拆分的工作可以随着迭代的发布周期同步上线，且不影响业务功能。落地时机的选择可以参考以下几种场景：

1、业务即将发生较大调整、演进或者核心业务变化频率较低时

考虑到拆分必要性和成本，如果业务基本不再变化，即便现在的服务有诸多问题，但运行稳定，拆分服务并没有太大的收益，所以时机上要选择业务演进仍在持续发生，且即将发生较大调整之前，原因在于基于当前的架构增加越多的业务逻辑，产生的耦合可能越大，未来拆分需要考虑的内容越多，难度就越大。这里的“较大调整”包括但不限于以下场景：

1、在服务的核心领域上做比较大规模的调整，比如重新调整业务流程进而适配市场变化

2、对服务的某个领域做大规模的增强以支持更多的需求，该领域的业务体量增大很多新增的系统集成，使架构上的依赖关系混乱

在产品迭代开发过程中实施拆分和新功能开发之间会存在一定的冲突。拆分工作一旦开始，对于新功能开发，有两种选择，一种是在拆分后基于新的架构进行，一种先按照原来的方式开发，再由负责拆分的同学把功能进行迁移，但具体采用什么策略，需要针对具体的问题具体分析，这样的功能越多，给拆分带来的额外工作量也越多。

2、有高并发、性能要求时

对于微服务化拆分后的服务，可以轻松地进行水平扩容，进行服务优化，满足更多的并发和性能需求。

在高并发场景下（或者资源紧张的场景下），我们希望一个请求如果不成功，不要占用资源，应该尽快失败，尽快返回，而且希望当一些边角的业务不正常的情况下，主要业务流程不受影响。这就需要熔断策略，也即当 A 调用 B，而 B 总是不正常的时候，为了让 B 不要波及到 A，可以对 B 的调用进行熔断，也即 A 不调用 B，而是返回暂时的 fallback 数据，当 B 正常的时候，再放开熔断，进行正常的调用。

如果核心业务流程和普通业务流程在同一个服务中，就需要使用大量的 if-else 语句，根据下发的配置来判断是否熔断或者降级，这会使得配置异常复杂，难以维护。如果核心业务和普通业务分成两个服务，就可以使用标准的熔断降级策略，配置在某种情况下，放弃对另一个服务的调用，可以进行统一的维护。

3、提交代码频繁出现大量冲突时

某一单体应用，多人开发，如果使用 GIT 做代码管理，则经常会遇到的事情就是代码提交冲突。当你想提交一个代码的时候，发现和别人提交的冲突了，于是因为你是后提交的人，你有责任去 merge 代码，好不容易 merge 成功了，等再次提交的时候，发现又冲突了。随着团队规模越大，冲突概率越大。

此时应该将服务拆分成不同的模块，每几人维护一个模块，也即一个工程，首先代码冲突的概率小多了，即使有了冲突，小组内基本上也能快速解决。每个模块对外提供接口，其他依赖模块可以不用关注具体的实现细节，只需要保证接口正确就可以。

4、当服务上线成为痛点时

未拆分微服务，每次上线都是一件很痛苦的事情。当你修改了一个小功能，但是你不能马上上线，因为你的服务依赖的其他模块还未开发完成，且更有可能依赖的模块还有一个依赖模块也在开发；当所有的模块都耦合在一起，互相依赖，谁也没办法独立上线，而是需要协调各个团队，大家开大会，约定一个时间点，无论大小功能，死活都要这天上线。

这种模式导致上线的时候，单次上线的需求列表非常长，这样风险比较大，可能小功能的错误会导致大功能的上线不正常，将如此长的功能，需要一点点 check，非常小心，这样上线时间长，影响范围大。

服务拆分后，在团队职责明确、应用边界明确、接口稳定的情况下，不同的模块可以独立上线。这样上线的次数增多，单次上线的需求列表变小，可以随时回滚，风险变小，时间变短，影响面小，从而迭代速度加快。对于接口要升级部分，保证灰度，先做接口新增，而非原接口变更，当注册中心中监控到的调用情况，发现接口已经不用了，再删除。

五、微服务拆分实践

未完待续...